

gRPC vs REST

Deep Dive Lecture Notes (Beginner to Advanced)

1. Introduction

Modern distributed applications are split into microservices. These services need to communicate with each other. Two of the most common communication styles are REST and gRPC.

2. What is REST?

REST (Representational State Transfer) is an architectural style. It treats everything as a resource. Clients manipulate resources using standard HTTP methods such as GET, POST, PUT and DELETE.

3. What is RPC?

RPC stands for Remote Procedure Call. Instead of thinking in terms of resources, you think in terms of methods. The client calls a method on another machine as though it were a local Java method.

4. What is gRPC?

gRPC is Google's high-performance RPC framework. It uses Protocol Buffers for serialization and HTTP/2 for transport. A .proto file defines services and messages. Code generators create client stubs and server skeletons automatically.

5. Resource vs Method Thinking

REST: GET /users/10, POST /orders, DELETE /users/5.

gRPC: getUser(), createOrder(), deleteUser().

6. Data Format

REST usually exchanges JSON, which is human-readable but larger. gRPC uses Protocol Buffers, a compact binary format that is smaller and faster to serialize and deserialize.

7. Communication Flow

REST: Client → HTTP Request → URL → Controller → JSON → Response.

gRPC: Client Stub → HTTP/2 → Protocol Buffers → Server Stub → Service Method → Response.

8. Connection Management

REST clients send independent HTTP requests. Modern clients often reuse TCP connections using keep-alive or pooling. gRPC typically creates a long-lived HTTP/2 channel. Multiple RPC calls

share that channel.

9. Multiplexing

HTTP/2 allows multiple independent request-response streams over a single TCP connection. This removes much of the waiting that occurs when many requests must be exchanged.

10. Streaming

REST is generally request-response. gRPC supports unary RPC, server streaming, client streaming and bidirectional streaming.

11. Service Discovery

Both REST and gRPC can integrate with service discovery such as Eureka. REST resolves a service name into a URL, while gRPC resolves the service name into a channel endpoint.

12. Security

Both commonly use TLS. Authentication mechanisms such as JWT or OAuth can be used with both.

13. Performance

gRPC generally performs better for internal microservice communication because of HTTP/2, multiplexing and Protocol Buffers. REST is easier for browsers, public APIs and third-party integrations.

14. REST vs gRPC Comparison

Aspect	REST	gRPC
API Style	Resource oriented	Method/RPC oriented
Transport	HTTP/1.1 or HTTP/2	HTTP/2
Payload	JSON (commonly)	Protocol Buffers
Human Readable	Yes	No (binary)
Performance	Good	Excellent
Streaming	Limited	Built-in
Browser Friendly	Excellent	Limited (needs gRPC-Web for browsers)
Code Generation	Manual DTOs/controllers	Automatic from .proto
Best Use	Public APIs	Internal microservices

15. Java Spring Boot Example

REST: Order Service uses RestTemplate or WebClient to call `http://notification-service/send`.

gRPC: Order Service creates a ManagedChannel and a generated Stub, then simply calls `stub.sendNotification(request)`.

16. When Should You Choose REST?

Choose REST for public APIs, browser applications, CRUD systems and third-party integrations.

17. When Should You Choose gRPC?

Choose gRPC for high-performance microservices, low latency communication, streaming applications and polyglot back-end systems.

18. Interview Questions

Q. Is REST an RPC?

No. REST is an architectural style, while RPC is a communication model.

Q. Does REST only use HTTP/1.1?

No. REST can use HTTP/1.1 or HTTP/2.

Q. Why is gRPC faster?

Protocol Buffers, HTTP/2, multiplexing and efficient code generation.

Q. Can REST and gRPC coexist?

Yes. Many systems expose REST externally and use gRPC internally.

19. Final Summary

REST focuses on resources and standard HTTP semantics. gRPC focuses on remote method invocation. REST usually exchanges JSON and is ideal for public APIs. gRPC uses Protocol Buffers and HTTP/2, making it an excellent choice for internal microservice communication where performance and streaming are important.